[1]:
```python
import pandas as pd
```

[2]:
```python
df = pd.read_csv('C:/Machine Learning/Machine Learning/Data Analysis/Notes/03-Pandas/mpg.csv')
```

[3]:
```python
df
```

[3]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 9 columns

[4]:
```python
year_cyl = df.groupby(['model_year','cylinders']).mean(numeric_only=True)
```

Suppose I want only model year for 1970:

[10]:
```python
year_cyl.xs(key=70,level='model_year')
```

[10]:

	mpg	displacement	weight	acceleration	origin
cylinders					
4	25.285714	107.000000	2292.571429	16.000000	2.285714
6	20.500000	199.000000	2710.500000	15.500000	1.000000
8	14.111111	367.555556	3940.055556	11.194444	1.000000

We can't pass a list to get multi-level:

[11]:
```python
year_cyl.xs(key=[70,80],level='model_year')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 year_cyl.xs(key=[70,80],level='model_year')

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\generic.py:4221, in NDFrame.xs(self, key, axis, level, drop_lev
el)
   4217         axis = self._get_axis_number(axis)
   4218         labels = self._get_axis(axis)
   4219 
   4220         if isinstance(key, list):
-> 4221             raise TypeError("list keys are not supported in xs, pass a tuple instead")
   4222 
   4223         if level is not None:
   4224             if not isinstance(labels, MultiIndex):

TypeError: list keys are not supported in xs, pass a tuple instead
```

Instead, do like this:

[12]:
```python
year_cyl.loc[[70,80]]
```

[12]:

		mpg	displacement	weight	acceleration	origin
model_year	**cylinders**					
70	4	25.285714	107.000000	2292.571429	16.000000	2.285714
	6	20.500000	199.000000	2710.500000	15.500000	1.000000
	8	14.111111	367.555556	3940.055556	11.194444	1.000000
80	3	23.700000	70.000000	2420.000000	12.500000	3.000000
	4	34.612000	111.000000	2360.080000	17.144000	2.200000
	5	36.400000	121.000000	2950.000000	19.900000	2.000000
	6	25.900000	196.500000	3145.500000	15.050000	2.000000

If we can use .loc, what's the use of cross-section? (.xs)

Suppose we want 4 cylinder cars, but for every single year, we can't use .loc, so it's 'cross-section' so we have to use '.xs'.

[17]:
```python
year_cyl.xs(key=4,axis=0,level='cylinders')
```

[17]:

	mpg	displacement	weight	acceleration	origin
model_year					
70	25.285714	107.000000	2292.571429	16.000000	2.285714
71	27.461538	101.846154	2056.384615	16.961538	1.923077
72	23.428571	111.535714	2382.642857	17.214286	1.928571
73	22.727273	109.272727	2338.090909	17.136364	2.000000
74	27.800000	96.533333	2151.466667	16.400000	2.200000
75	25.250000	114.833333	2489.250000	15.833333	2.166667
76	26.766667	106.333333	2306.600000	16.866667	1.866667
77	29.107143	106.500000	2205.071429	16.064286	1.857143
78	29.576471	112.117647	2296.764706	16.282353	2.117647
79	31.525000	113.583333	2357.583333	15.991667	1.583333
80	34.612000	111.000000	2360.080000	17.144000	2.200000
81	32.814286	108.857143	2275.476190	16.466667	2.095238
82	32.071429	118.571429	2402.321429	16.703571	1.714286

[16]:
```python
help(year_cyl.xs)
```

```
Help on method xs in module pandas.core.generic:

xs(
    key: IndexLabel,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    drop_level: bool = True
) -> Self method of pandas.DataFrame instance
    Return cross-section from the Series/DataFrame.

    This method takes a `key` argument to select data at a particular
    level of a MultiIndex.

    Parameters
    ----------
    key : label or tuple of label
        Label contained in the index, or partially in a MultiIndex.
    axis : {0 or 'index', 1 or 'columns'}, default 0
        Axis to retrieve cross-section on.
    level : object, defaults to first n levels (n=1 or len(key))
        In case of a key partially contained in a MultiIndex, indicate
        which levels are used. Levels can be referred by label or position.
    drop_level : bool, default True
        If False, returns object with same levels as self.

    Returns
    -------
    Series or DataFrame
        Cross-section from the original Series or DataFrame
        corresponding to the selected index levels.

    See Also
    --------
    DataFrame.loc : Access a group of rows and columns
        by label(s) or a boolean array.
    DataFrame.iloc : Purely integer-location based indexing
        for selection by position.

    Notes
    -----
    `xs` can not be used to set values.

    MultiIndex Slicers is a generic way to get/set values on
    any level or levels.
    It is a superset of `xs` functionality, see
    :ref:`MultiIndex Slicers <advanced.mi_slicers>`.

    Examples
    --------
    >>> d = {
    ...     "num_legs": [4, 4, 2, 2],
    ...     "num_wings": [0, 0, 2, 2],
    ...     "class": ["mammal", "mammal", "mammal", "bird"],
    ...     "animal": ["cat", "dog", "bat", "penguin"],
    ...     "locomotion": ["walks", "walks", "flies", "walks"],
    ... }
    >>> df = pd.DataFrame(data=d)
    >>> df = df.set_index(["class", "animal", "locomotion"])
    >>> df
                               num_legs  num_wings
    class  animal  locomotion
    mammal cat     walks              4          0
           dog     walks              4          0
           bat     flies              2          2
    bird   penguin walks              2          2

    Get values at specified index

    >>> df.xs("mammal")
                       num_legs  num_wings
    animal locomotion
    cat    walks              4          0
    dog    walks              4          0
    bat    flies              2          2

    Get values at several indexes

    >>> df.xs(("mammal", "dog", "walks"))
    num_legs     4
    num_wings    0
    Name: (mammal, dog, walks), dtype: int64

    Get values at specified index and level

    >>> df.xs("cat", level=1)
                       num_legs  num_wings
    class  locomotion
    mammal walks              4          0

    Get values at several indexes and levels

    >>> df.xs(("bird", "walks"), level=[0, "locomotion"])
             num_legs  num_wings
    animal
    penguin         2          2

    Get values at specified column and axis

    >>> df.xs("num_wings", axis=1)
    class   animal   locomotion
    mammal  cat      walks         0
            dog      walks         0
            bat      flies         2
    bird    penguin  walks         2
    Name: num_wings, dtype: int64
```

Suppose we want 6 and 8 of all sections together, since we can't pass a list in .xs, the best way to do is:

First filter out the 6 and 8 values from the original dataframe and then use groupby()

[18]:
```python
df[df['cylinders'].isin([6,8])]
```

[18]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
365	20.2	6	200.0	88	3060	17.1	81	1	ford granada gl
366	17.6	6	225.0	85	3465	16.6	81	1	chrysler lebaron salon
386	25.0	6	181.0	110	2945	16.4	82	1	buick century limited
387	38.0	6	262.0	85	3015	17.0	82	1	oldsmobile cutlass ciera (diesel)
389	22.0	6	232.0	112	2835	14.7	82	1	ford granada l

187 rows × 9 columns

[25]:
```python
df[df['cylinders'].isin([6,8])].groupby(['model_year','cylinders']).mean(numeric_only=True)
```

[25]:

		mpg	displacement	weight	acceleration	origin
model_year	**cylinders**					
70	6	20.500000	199.000000	2710.500000	15.500000	1.000000
	8	14.111111	367.555556	3940.055556	11.194444	1.000000
71	6	18.000000	243.375000	3171.875000	14.750000	1.000000
	8	13.428571	371.714286	4537.714286	12.214286	1.000000
72	8	13.615385	344.846154	4228.384615	13.000000	1.000000
73	6	19.000000	212.250000	2917.125000	15.687500	1.250000
	8	13.200000	365.250000	4279.050000	12.250000	1.000000
74	6	17.857143	230.428571	3320.000000	16.857143	1.000000
	8	14.200000	315.200000	4438.400000	14.700000	1.000000
75	6	17.583333	233.750000	3398.333333	17.708333	1.000000
	8	15.666667	330.500000	4108.833333	13.166667	1.000000
76	6	20.000000	221.400000	3349.600000	17.000000	1.300000
	8	14.666667	324.000000	4064.666667	13.222222	1.000000
77	6	19.500000	220.400000	3383.000000	16.900000	1.400000
	8	16.000000	335.750000	4177.500000	13.662500	1.000000
78	6	19.066667	213.250000	3314.166667	16.391667	1.166667
	8	19.050000	300.833333	3563.333333	13.266667	1.000000
79	6	22.950000	205.666667	3025.833333	15.433333	1.000000
	8	18.630000	321.400000	3862.900000	15.400000	1.000000
80	6	25.900000	196.500000	3145.500000	15.050000	2.000000
81	6	23.428571	184.000000	3093.571429	15.442857	1.714286
	8	26.600000	350.000000	3725.000000	19.000000	1.000000
82	6	28.333333	225.000000	2931.666667	16.033333	1.000000

Let's see how to sort the multi-level index:

Now, it's sorted from earliest model_year to latest model_year, let's try reverse:

[26]:
```python
year_cyl.sort_index(level='model_year',ascending=False)
```

[26]:

		mpg	displacement	weight	acceleration	origin
model_year	**cylinders**					
82	6	28.333333	225.000000	2931.666667	16.033333	1.000000
	4	32.071429	118.571429	2402.321429	16.703571	1.714286
81	8	26.600000	350.000000	3725.000000	19.000000	1.000000
	6	23.428571	184.000000	3093.571429	15.442857	1.714286
	4	32.814286	108.857143	2275.476190	16.466667	2.095238
80	6	25.900000	196.500000	3145.500000	15.050000	2.000000
	5	36.400000	121.000000	2950.000000	19.900000	2.000000
	4	34.612000	111.000000	2360.080000	17.144000	2.200000
	3	23.700000	70.000000	2420.000000	12.500000	3.000000
79	8	18.630000	321.400000	3862.900000	15.400000	1.000000
	6	22.950000	205.666667	3025.833333	15.433333	1.000000
	5	25.400000	183.000000	3530.000000	20.100000	2.000000
	4	31.525000	113.583333	2357.583333	15.991667	1.583333
78	8	19.050000	300.833333	3563.333333	13.266667	1.000000
	6	19.066667	213.250000	3314.166667	16.391667	1.166667
	5	20.300000	131.000000	2830.000000	15.900000	2.000000
	4	29.576471	112.117647	2296.764706	16.282353	2.117647
77	8	16.000000	335.750000	4177.500000	13.662500	1.000000
	6	19.500000	220.400000	3383.000000	16.900000	1.400000
	4	29.107143	106.500000	2205.071429	16.064286	1.857143
	3	21.500000	80.000000	2720.000000	13.500000	3.000000
76	8	14.666667	324.000000	4064.666667	13.222222	1.000000
	6	20.000000	221.400000	3349.600000	17.000000	1.300000
	4	26.766667	106.333333	2306.600000	16.866667	1.866667
75	8	15.666667	330.500000	4108.833333	13.166667	1.000000
	6	17.583333	233.750000	3398.333333	17.708333	1.000000
	4	25.250000	114.833333	2489.250000	15.833333	2.166667
74	8	14.200000	315.200000	4438.400000	14.700000	1.000000
	6	17.857143	230.428571	3320.000000	16.857143	1.000000
	4	27.800000	96.533333	2151.466667	16.400000	2.200000
73	8	13.200000	365.250000	4279.050000	12.250000	1.000000
	6	19.000000	212.250000	2917.125000	15.687500	1.250000
	4	22.727273	109.272727	2338.090909	17.136364	2.000000
	3	18.000000	70.000000	2124.000000	13.500000	3.000000
72	8	13.615385	344.846154	4228.384615	13.000000	1.000000
	4	23.428571	111.535714	2382.642857	17.214286	1.928571
	3	19.000000	70.000000	2330.000000	13.500000	3.000000
71	8	13.428571	371.714286	4537.714286	12.214286	1.000000
	6	18.000000	243.375000	3171.875000	14.750000	1.000000
	4	27.461538	101.846154	2056.384615	16.961538	1.923077
70	8	14.111111	367.555556	3940.055556	11.194444	1.000000
	6	20.500000	199.000000	2710.500000	15.500000	1.000000
	4	25.285714	107.000000	2292.571429	16.000000	2.285714

[27]:
```python
df
```

[27]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 9 columns

What we earlier did is specifying a same function for all the values, but in real life, it doesn't really makes sense, what we can do is, let's try giving separate aggregate functions to each and every column possible:

It's in old pandas:

[31]:
```python
df.agg(['std', 'mean'])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[31], line 1
----> 1 df.agg(['std', 'mean'])

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\frame.py:12046, in DataFrame.aggregate(self, func, axis, *args,
**kwargs)
  12042
  12043         axis = self._get_axis_number(axis)
  12044
  12045         op = frame_apply(self, func=func, axis=axis, args=args, kwargs=kwargs)
> 12046         result = op.agg()
  12047         result = reconstruct_and_relabel_result(result, func, **kwargs)
  12048         return result

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:1027, in FrameApply.agg(self)
   1025 result = None
   1026 try:
-> 1027     result = super().agg()
   1028 finally:
   1029     self.obj = obj

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:297, in Apply.agg(self)
    294     return self.agg_dict_like()
    295 elif is_list_like(func):
    296     # we require a list, but not a 'str'
--> 297     return self.agg_list_like()
    299 # caller can react
    300 return None

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:414, in Apply.agg_list_like(self)
    406 def agg_list_like(self) -> DataFrame | Series:
    407     """
    408     Compute aggregation in the case of a list-like argument.
    409
   (...)    412     Result of aggregation.
    413     """
--> 414     return self.agg_or_apply_list_like(op_name="agg")

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:843, in NDFrameApply.agg_or_apply_list_like(self, op_n
ame)
    840 if getattr(obj, "axis", 0) == 1:
    841     raise NotImplementedError("axis other than 0 is not supported")
--> 843 keys, results = self.compute_list_like(op_name, obj, kwargs)
    844 result = self.wrap_results_list_like(keys, results)
    845 return result

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:473, in Apply.compute_list_like(self, op_name, selecte
d_obj, kwargs)
    467 colg = obj._gotitem(col, ndim=1, subset=selected_obj.iloc[:, index])
    468 args = (
    469     [self.axis, *self.args]
    470     if include_axis(op_name, colg)
    471     else self.args
    472 )
--> 473 new_res = getattr(colg, op_name)(func, *args, **kwargs)
    474 results.append(new_res)
    475 indices.append(index)

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\series.py:4808, in Series.aggregate(self, func, axis, *args, **
kwargs)
   4805     func = dict(kwargs.items())
   4807 op = SeriesApply(self, func, args=args, kwargs=kwargs)
-> 4808 result = op.agg()
   4809 return result

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:1523, in SeriesApply.agg(self)
   1522 def agg(self):
-> 1523     result = super().agg()
   1524     if result is None:
   1525         obj = self.obj

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:297, in Apply.agg(self)
    294     return self.agg_dict_like()
    295 elif is_list_like(func):
    296     # we require a list, but not a 'str'
--> 297     return self.agg_list_like()
    299 # caller can react
    300 return None

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:414, in Apply.agg_list_like(self)
    406 def agg_list_like(self) -> DataFrame | Series:
    407     """
    408     Compute aggregation in the case of a list-like argument.
    409
   (...)    412     Result of aggregation.
    413     """
--> 414     return self.agg_or_apply_list_like(op_name="agg")

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:843, in NDFrameApply.agg_or_apply_list_like(self, op_n
ame)
    840 if getattr(obj, "axis", 0) == 1:
    841     raise NotImplementedError("axis other than 0 is not supported")
--> 843 keys, results = self.compute_list_like(op_name, obj, kwargs)
    844 result = self.wrap_results_list_like(keys, results)
    845 return result

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:457, in Apply.compute_list_like(self, op_name, selecte
d_obj, kwargs)
    451 colg = obj._gotitem(selected_obj.name, ndim=1, subset=selected_obj)
    452 args = (
    453     [self.axis, *self.args]
    454     if include_axis(op_name, colg)
    455     else self.args
    456 )
--> 457 new_res = getattr(colg, op_name)(a, *args, **kwargs)
    458 results.append(new_res)
    460 # make sure we find a good name

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\series.py:4808, in Series.aggregate(self, func, axis, *args, **
kwargs)
   4805     func = dict(kwargs.items())
   4807 op = SeriesApply(self, func, args=args, kwargs=kwargs)
-> 4808 result = op.agg()
   4809 return result

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:1523, in SeriesApply.agg(self)
   1522 def agg(self):
-> 1523     result = super().agg()
   1524     if result is None:
   1525         obj = self.obj

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:291, in Apply.agg(self)
    288 func = self.func
    290 if isinstance(func, str):
--> 291     return self.apply_str()
    293 if is_dict_like(func):
    294     return self.agg_dict_like()

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:701, in Apply.apply_str(self)
    697     if "axis" in arg_names and not isinstance(
    698         obj, (SeriesGroupBy, DataFrameGroupBy)
    699     ):
    700         self.kwargs["axis"] = self.axis
--> 701 return self._apply_str(obj, func, *self.args, **self.kwargs)

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\apply.py:792, in Apply._apply_str(self, obj, func, *args, **kwa
rgs)
    790 f = getattr(obj, func)
    791 if callable(f):
--> 792     return f(*args, **kwargs)
    794 # people may aggregate on a non-callable attribute
    795 # but don't let them think they can pass args to it
    796 assert len(args) == 0

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\util\_decorators.py:336, in deprecate_nonkeyword_arguments.<locals
>.decorate.<locals>.wrapper(*args, **kwargs)
    330 if len(args) > num_allow_args:
    331     warnings.warn(
    332         msg.format(arguments=_format_argument_list(allow_args)),
    333         klass,
    334         stacklevel=find_stack_level(),
    335     )
--> 336 return func(*args, **kwargs)

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\series.py:8401, in Series.std(self, axis, skipna, ddof, numeric
_only, **kwargs)
   8346 @deprecate_nonkeyword_arguments(Pandas4Warning, allowed_args=["self"], name="std")
   8347 def std(
   8348     self,
   (...)   8353     **kwargs,
   8354 ):
   8355     """
   8356     Return sample standard deviation.
   8357
   (...)   8399     0.816496580927726
   8400     """
-> 8401     return NDFrame.std(
   8402         self,
   8403         axis=axis,
   8404         skipna=skipna,
   8405         ddof=ddof,
   8406         numeric_only=numeric_only,
   8407         **kwargs,
   8408     )

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\generic.py:11766, in NDFrame.std(self, axis, skipna, ddof, nume
ric_only, **kwargs)
  11762         ddof: int = 1,
  11763         numeric_only: bool = False,
  11764         **kwargs,
  11765     ) -> Series | float:
> 11766         return self._stat_function_ddof(
  11767             "std", nanops.nanstd, axis, skipna, ddof, numeric_only, **kwargs
  11768         )

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\generic.py:11727, in NDFrame._stat_function_ddof(self, name, fu
nc, axis, skipna, ddof, numeric_only, **kwargs)
  11723     ) -> Series | float:
  11724         nv.validate_stat_ddof_func((), kwargs, fname=name)
  11725         validate_bool_kwarg(skipna, "skipna", none_allowed=False)
  11726
> 11727         return self._reduce(
  11728             func, name, axis=axis, numeric_only=numeric_only, skipna=skipna, ddof=ddof
  11729         )

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\series.py:7480, in Series._reduce(self, op, name, axis, skipna,
numeric_only, filter_type, **kwds)
   7476     self._get_axis_number(axis)
   7478 if isinstance(delegate, ExtensionArray):
   7479     # dispatch to ExtensionArray interface
-> 7480     result = delegate._reduce(name, skipna=skipna, **kwds)
   7482 else:
   7483     # dispatch to numpy arrays
   7484     if numeric_only and self.dtype.kind not in "iufcb":
   7485         # i.e. not is_numeric_dtype(self.dtype)

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\arrays\string_.py:969, in StringArray._reduce(self, name, skipn
a, keepdims, axis, **kwargs)
    966         return self._from_sequence([result], dtype=self.dtype)
    967     return result
--> 969 raise TypeError(f"Cannot perform reduction '{name}' with string dtype")

TypeError: Cannot perform reduction 'std' with string dtype
```

In new Pandas:

[32]:
```python
df.select_dtypes(include='number').agg(['std','mean'])
```

[32]:

	mpg	cylinders	displacement	weight	acceleration	model_year	origin
std	7.815984	1.701004	104.269838	846.841774	2.757689	3.697627	0.802055
mean	23.514573	5.454774	193.425879	2970.424623	15.568090	76.010050	1.572864

[33]:
```python
df.select_dtypes(include='number').agg(['std','mean'])['mpg']
```

[33]:
```
std      7.815984
mean    23.514573
Name: mpg, dtype: float64
```

[34]:
```python
df.select_dtypes(include='number').agg({'mpg':'std','cylinders':'mean'})
```

[34]:
```
mpg          7.815984
cylinders    5.454774
dtype: float64
```

If multiple operations on a single column:

[35]:
```python
df.select_dtypes(include='number').agg({'mpg':['max','mean']})
```

[35]:

	mpg
max	46.600000
mean	23.514573

Why does the first agg() result appear as a Series, but the second agg() result appear as a DataFrame? How does Pandas decide which type to return?

In first case, each column has one operation, so Pandas returns a Series. And, a series is displayed as a simple list-like output.

Now, mpg has multiple operations (max and mean). Pandas needs a table to store multiple results, so it returns a dataframe.

[37]:
```python
df1 = df.select_dtypes(include='number').agg({'mpg': ['max','mean'],'weight': ['min','std']})
```

[44]:
```python
df1.fillna('Not calculated').transpose()
```

[44]:

	max	mean	min	std
mpg	46.6	23.514573	Not calculated	Not calculated
weight	Not calculated	Not calculated	1613.0	846.841774